

1. Project Overview

This project involves the development of an autonomous drone navigation system within the Gazebo simulation environment. The objective is to enable the drone to detect, approach, and stabilize over a target (QR/AR code) and extract encoded information.

2. Technical Approach

- **Environment & Simulation:** Gazebo (Jazzy/Ignition) is used as the primary simulation engine. The drone is controlled via ROS 2 topics.
- **Control Logic:** A proportional-derivative-like approach is used to calculate linear and angular velocities based on target error. The drone maintains a set hover height and adjusts its X/Y/Yaw coordinates to remain centered on the target.
- **Vision Pipeline:**
 - **Pre-processing:** Grayscale conversion and adaptive thresholding to isolate geometric shapes.
 - **Detection:** Contours analysis to find quadrilateral shapes representing the target.
 - **PnP (Perspective-n-Point):** `cv2.solvePnP` is used to estimate the drone's 3D pose relative to the target for height calculation.
 - **Decoding:** `pyzbar` and `cv2.QRCodeDetector` are utilized to extract the encoded string once the drone achieves a "stable" state.

3. Key Libraries Used

- **ROS 2 (Jazzy):** For robot communication, subscriber/publisher patterns, and system integration.
- **OpenCV (cv2):** For real-time computer vision, frame processing, and PnP pose estimation.
- **Pyzbar:** For robust QR code decoding.
- **NumPy:** For high-speed matrix calculations required for camera matrices and PnP.
- **CvBridge:** To bridge ROS 2 image messages with OpenCV-compatible formats.

4.How-to-use:

- Install all requirements in requirements.txt
- Run run.sh
- It will auto run the bridge and open gazebo with the world.
- Spawn the Parrot Bebop 2 drone used as a placeholder drone into the gazebo world with the resource spawner. The drone is already downloaded and accessible in local resources in Gazebo
- Place the drone at a height above the QR code
- In a new terminal, run:
 - source /opt/ros/jazzy/setup.bash , or source to the local ros installation
 - python3 qr_tracker.py, to run the actual tracker software.

4.Documentation&visuals:

Github Repo:

https://github.com/CODERX-beta/Egl_Vsn

Running Screenshot on Ubuntu 24.04 with Gazebo jazzy and ROS Jetty:

